graph-rag: CPG-Based Vulnerability Retrieval

Technical Progress Summary for Stakeholders

Master's Thesis — TU Munich — Siemens AI Engineering

Motivation & Problem Statement

Identifying whether a new code snippet contains a known vulnerability pattern requires searching across thousands of historical CVEs efficiently. Classical text search misses structural code patterns; pure LLM-based approaches lack precise structural grounding. This project builds a **retrieval-augmented generation (RAG) system indexed on Code Property Graphs (CPGs)**, enabling similarity search at the level of program structure rather than surface text.

Data Pipeline

Currently Implemented

1. **CPG Export via Joern.** C/C++ source functions are parsed with `joern-parse` and exported as GraphML using `joern-export --repr all`. Each function produces a directory of `export.xml` files covering AST, CFG, CDG, and PDG edges.
2. **Graph Loading & Merging.** All `export.xml` files under a function directory are merged into a single `MultiDiGraph` (NetworkX). Phantom nodes (auto-created by edge references to missing nodes) are identified by tracking declared vs. referenced node IDs and removed. COMMENT and UNKNOWN nodes are filtered.
3. **Semantic Graph Diff.** Vulnerable (*before*) and patched (*after*) CPGs are aligned using **fingerprint-based node matching** — nodes are matched by `(labelV, normalised CODE)` rather than Joern-assigned IDs, which are non-deterministic across runs. Changed nodes are classified as `removed`, `added`, `mutated` (same structure, different code), or `rewired` (edge topology changed). Each node in the vulnerability-relevant subgraph (G_{vuln}) receives a `diff_weight` $\in [0, 1]$ encoding its change significance.
4. **Dataset Loaders.** Three datasets are supported via a common `BaseDataset` interface that yields `FunctionPair` objects:
   - **BigVul** — CSV with `func_before`/`func_after` columns.
   - **CVEFixes** — SQLite database with `method_change` table.
   - **AutoPatch** — folder-per-CVE with original source files and LLM-generated augmented variants (augmented, re-implemented by GPT-4o, DeepSeek, LLaMA, o3-mini).

   All loaders share a `export_jobs()` method that feeds the Joern export step, and a `stream()` method for index building.
5. **Multiprocessing Joern Export.** `main.py --mode export` parallelises Joern invocations across functions using a `multiprocessing.Pool`. Skip-if-exists logic makes re-runs safe after partial failures.
6. **Embedding & Indexing.** Extracted graphs are embedded and stored in a FAISS index with aligned JSON metadata (CVE ID, CWE, function name, dataset, variant).
7. **Experiment Framework.** A runner evaluates all `(embedder, backend, graph_variant)` combinations and computes self-retrieval, leave-one-out, CWE-group recall, query latency, and embedding space statistics. Results are exported to JSON and visualised in a standalone HTML dashboard.

Next Steps

1. **PDG density validation.** Confirm that `REACHING_DEF` and `CDG` edges are present in exports (`--repr all` flag). Currently $> 50\%$ of pairs fall back to CFG-only slicing.

2. **Ground-truth query labels.** Construct a held-out query set with known CVE targets for honest precision/recall evaluation.
3. **Temporal generalisation split.** Train on CVEs before year Y, query with year $Y+1$ CVEs to measure generalisation to unseen patterns.
4. **Agent integration.** Wire the `Retriever` into ADK agents for end-to-end patch suggestion.
5. **Scaling.** Extend to full BigVul (∼3,500 CVEs) and CVEFixes (∼10,000 method pairs).

RAG Index Backends

1. **FAISS Flat (IndexFlatIP).** Exact inner-product search over L2-normalised vectors (equivalent to cosine similarity). No approximation; guarantees correct top-k. Scales to ∼100k vectors on CPU without issue. Used as the correctness baseline in all experiments.
2. **HNSW (Hierarchical Navigable Small World).** Approximate nearest-neighbour index via a layered proximity graph. Sub-linear query time $O(\log n)$ vs. linear for flat search. Configurable via `M` (graph connectivity) and `ef_construction`/`ef_search` (recall/speed trade-off). Preferred backend for production at >10k vectors. Both backends share an identical interface (`add`, `save`, `load`) and are interchangeable in config.

Graph Extraction Strategies

1. **ID-based diff (baseline, deprecated).** Original approach: set difference of Joern node IDs between *before* and *after* graphs. Produces $G_{\text{vuln}} \approx G_{\text{before}}$ because Joern assigns different IDs to semantically identical nodes across runs.
2. **Semantic diff with n-hop expansion.** Fingerprint-matched diff with configurable neighbourhood expansion ($n \in \{0, 1, 2, 3\}$ hops). $n = 0$ returns only directly changed nodes; $n = 3$ adds structural context. Experiments show $n = 1$ or $n = 2$ best balances signal density and context.
3. **CFG-only subgraph.** Retains only control-flow edges and their endpoints. Reduces AST noise (AST edges dominate most CPGs) and focuses on execution path structure.
4. **PDG slice (S1).** Follows `REACHING_DEF`, `CDG`, and `REF` edges from changed nodes via bidirectional BFS. Captures data and control dependencies — theoretically the most semantically rich representation for vulnerability detection (Yamaguchi et al. 2014, Ferrante et al. 1987). Falls back to CFG automatically when PDG edge density < 10%.
5. **Change-anchored PDG hop (S2).** Restricts n-hop expansion to semantic edge types only (`CFG` ∪ `CDG` ∪ `REACHING_DEF`). Eliminates AST-dominated context that inflates G_{vuln} without adding vulnerability signal.
6. **Vulnerability pattern motifs (S3).** Counts occurrences of 8 hand-crafted 3-node motifs around changed nodes (e.g. Call→Identifier via `REACHING_DEF` for use-after-free; Call→ControlStructure via `CFG` for missing checks). Returns a normalised 8-dimensional histogram — CWE-agnostic and independent of graph topology.

Embedders

All embedders are **unsupervised** (no labelled training data required) and output L2-normalised vectors of configurable dimension (default 128).

1. **NetLSD** (*Network Laplacian Spectral Descriptor*). Computes the heat trace of the normalised graph Laplacian across a log-spaced timescale range. Permutation-invariant; captures global spectral structure. Sensitive to graph size — collapses on graphs with < 3 nodes. Weighted variant uses `diff_weight` to bias edge weights toward changed nodes.

2. **WL Baseline** (*Weisfeiler-Lehman*). Iterative colour refinement over node types followed by weighted sum pooling and a linear projection. Equivalent in expressive power to graph2vec's subtree kernel but implemented in PyTorch Geometric with no external dependency. `diff_weight` biases pooling toward high-signal nodes.
3. **GIN** (*Graph Isomorphism Network*). 3-layer GIN with random (frozen) weights. Random projections preserve pairwise distances (Johnson–Lindenstrauss) and provide stronger structural fingerprints than WL on small graphs due to MLP non-linearity. Dual pooling (sum + mean concatenated).
4. **Combined.** Concatenates NetLSD + WL + GIN descriptors ($3 \times 128 = 384$ dimensions) and reduces via PCA to the target dimension. Explained variance reported at fit time. Most robust to individual embedder collapse.
5. **Motif Embedder.** Concatenates: (i) motif histogram (8-dim), (ii) semantic CODE features (ptr/alloc/free/lock/cast counts per node, 12-dim), (iii) node and edge type histograms (15-dim), (iv) NetLSD on PDG slice (16-dim). Total ∼51 raw features projected to target dimension via Gaussian random projection (no training). Designed to be discriminative when pure topology is not.

Experiments & Early Results

Experiment Grid

All experiments evaluate the full `(embedder, backend, graph_variant)` grid. Metrics: **Hit@K** (fraction of queries where the correct CVE appears in top-K results), **MRR** (mean reciprocal rank), **CWE-group recall** (fraction of top-K results sharing the query's CWE type), **effective dimensionality** of the embedding space, and **query latency** (p50/p99).

Graph Strategy Experiments (E1–E8)

- **E1** — diff with $n \in \{0, 1, 2, 3\}$ hops.
- **E2** — subgraph strategies: CFG-only, PDG slice, method-entry BFS, full G_{before}.
- **E3** — CFG topological order as node feature.
- **E4** — semantic CODE token features on G_{vuln}.
- **E5** — pure semantic vector (no topology).
- **E6** — hybrid: PDG slice + semantic features.
- **E7** — PDG slice + motif embedder.
- **E8** — motif histogram on PDG slice.

Results Summary

Preliminary findings on BigVul subset (∼200 CVEs):

- PDG slice (S1) with GIN+NetLSD achieves **Hit@10 = 0.68**.
- Motif embedder (S3) is most robust to graph collapse; suitable for production at scale.
- HNSW (M=16, ef=100) achieves **<5ms p99 latency** with <2% recall loss vs. FAISS flat.
- CWE-group recall reaches 0.55, indicating structural patterns are indeed recoverable.

Next Milestones

1. Scale to full BigVul and CVEFixes.
2. Integrate with ADK agent framework.
3. Temporal split evaluation.
4. Dashboard visualisation and open-source release.

This is a **bounded BFS (breadth-first search) program slice** starting from the seed nodes identified in steps 1–2.

What it does: Starting from every node in `changed` (removed nodes, fix-adjacent nodes, edge-changed nodes), it expands outward up to $SLICE_DEPTH = 3$ hops, but only along flow edges: `CFG`, `CDG`, `REACHING_DEF`, `PDG`, `DDG`. It follows edges in both directions (successors via `out_edges` and predecessors via `in_edges`).

Why: The `changed` set alone only contains the directly modified nodes. But a vulnerability's behavior depends on surrounding control/data-flow context — e.g. a use-after-free involves a `free()` call (changed) **and** the subsequent `use()` (unchanged but reachable via `REACHING_DEF`).

The 3-hop expansion captures that neighborhood while staying bounded.

Walk-through of one iteration: 1. `frontier` starts as all `changed` nodes 2. For each node in `frontier`, collect all neighbors reachable by one flow edge that aren't already in `slice_nodes` → that's `next_frontier` 3. Add `next_frontier` to `slice_nodes`, then repeat with `next_frontier` as the new frontier 4. After 3 rounds, `slice_nodes` contains everything within 3 flow-edge hops of any changed node

The result is the vulnerability-relevant **program slice** — the subgraph of `G_before` that is transitively connected to the patch through data/control dependencies, bounded to avoid pulling in the entire CPG.

Experiments

Dataset and Evaluation Protocol

All experiments are conducted on the **AutoPatch** dataset, comprising 75 original Linux kernel CVE functions and their LLM-generated re-implementations (5 variants per CVE: augmented, GPT-4o, DeepSeek, DeepSeek-R1, LLaMA, o3-mini), yielding **225 index pairs** after filtering pairs with failed Joern exports.

We use a **stratified train/test split** (seed = 42, test ratio = 0.2) that partitions the augmented variants by CWE class to ensure each class is represented in both sets. All 75 original CVE pairs are retained in the index; **73 held-out augmented pairs** form the query set. At query time, the system must retrieve a same-CVE item from the 225-pair index for each of the 73 queries.

Metrics. We report the following, all computed at k=5 unless stated otherwise:

- **hit@k**: fraction of queries whose correct CVE appears in the top-k retrieved results (micro-averaged).
- **MRR**: mean reciprocal rank of the first correct CVE across all queries.
- **CWE Recall**: for each query, the fraction of same-CWE items retrieved in top-k, macro-averaged across CWE classes.
- **Effective dimensionality**: the number of PCA components needed to explain 90% of the embedding variance, measuring how many independent directions the embedder uses.

Embedders. Six unsupervised embedders are evaluated: three *structural* (Combined = NetLSD ‖ WL ‖ GIN concatenated and PCA-projected to 128-d; GIN alone; WL alone), two *semantic* (CodeBERT-seq using the `[CLS]` token of changed-node code; CodeBERT-pattern adding 34 structural features), and one *hybrid* (RGCN, a relational GCN with edge-type-specific weight matrices). All embedders output L2-normalised 128-dimensional vectors.

Role of PCA. Several embedders apply Principal Component Analysis (PCA) as a post-processing step to reduce the raw descriptor dimensionality to the target 128 dimensions. This is motivated by two complementary considerations. First, in high-dimensional spaces pairwise distances *concentrate*: the ratio of maximum to minimum distance among a set of points converges to 1 as dimensionality grows, making nearest-neighbour discrimination unreliable [1]. Projecting to a lower-dimensional subspace alleviates this effect by discarding dimensions that carry mostly noise. Second, PCA selects the linear subspace that maximises preserved variance (equivalently, minimises mean squared reconstruction error) [2], ensuring that the most discriminative directions are retained. For the Combined embedder, whose 384-dimensional input is a concatenation of three heterogeneous sub-embedders (spectral, colour-refinement, and message-passing), PCA additionally decorrelates redundant features across sub-embedders and produces a compact, jointly informative representation. For CodeBERT (768→128), PCA serves primarily as a dimensionality equaliser so that all embedders share the same vector size for fair comparison in the FAISS index. Raunak et al. [3] showed empirically that PCA post-processing of pre-trained embeddings improves downstream task performance by

removing dominated variance directions that encode frequency rather than semantics—an effect we observe in our effective-dimensionality analysis (Section 8.2).

PCA is always **fitted on the index set only**; query vectors are projected through the fitted transform without updating it, following standard information-retrieval practice to prevent test-set leakage.

Graph Ablation

The central question of this experiment is whether *vulnerability-focused graph slicing* and *diff annotations* improve retrieval over using the full pre-patch CPG. We define a 2×2 factorial design crossing two factors:

1. **Slicing**: *full* (the complete pre-patch CPG, G_{before}) vs. *sliced* (the fingerprint-diff subgraph G_{vuln}, retaining only nodes within 3 flow-edge hops of changed nodes).
2. **Diff labelling**: *absent* (all `diff` and `diff_weight` node attributes stripped) vs. *present* (each node annotated with its change class and a weight $w \in \{1.0, 0.8, 0.6, 0.2\}$ for `removed`, `fix_adjacent`, `edge_changed`, and `context` respectively).

This yields four graph representations:

Symbol	Sliced?	Labels?	Construction
G_{before}	No	No	Full pre-patch CPG; diff attributes removed.
$G_{\text{before}}^{+\ell}$	No	Yes	Full pre-patch CPG; diff labels transferred from G_{vuln}; unmatched nodes default to `context` (w=0
$G_{\text{vuln}}^{-\ell}$	Yes	No	Fingerprint-sliced subgraph; diff attributes stripped.
G_{vuln}	Yes	Yes	Fingerprint-sliced subgraph with native diff labels and weights.

8.2.1 Experimental Configurations

The ablation was run under three configurations that differ in *query protocol* and *PCA fitting procedure*, allowing us to disentangle representation quality from methodological artefacts.

	Run A	Run B	Run C
Script	`slicing_comparison`	`slicing_comparison`	`runner.py`
Query protocol	Symmetric: query graph uses the same variant as the index.	Asymmetric: augmented queries use G_{before}; original queries use G_{vuln}.	Same as Run B.
PCA fitting	Index-only: `embed_many` on index, then on queries (PCA reused).	Same as Run A.	Index-only: PCA fitted via `embed_many` on index; queries projected via `embed_one`.
Graph variants	All four.	All four.	G_{vuln} only.

Run A tests each graph representation in isolation (query and index always use the same variant), measuring the intrinsic discriminative power of each representation. Runs B and C use the *asymmetric* protocol that reflects a realistic deployment scenario: augmented (LLM-rewritten) queries lack a patched version and can only provide the full pre-patch CPG, while original CVE queries have access to the vulnerability-sliced subgraph. Runs B and C differ only in the embedding call path (`embed_many` vs. `embed_one` for queries), allowing a direct comparison of batch vs. single-query embedding under identical data and PCA protocol.

8.2.2 Results

Table 1 presents the full factorial results for Run A (symmetric protocol). Table 2 compares the G_{vuln} condition across all three runs.

Table 1: Run A (symmetric protocol): hit@1 (%) across all graph variants and embedders. 225 index, 73 queries, seed = 42.

Embedder	G_{before}	$G^{+\ell}_{\text{before}}$	$G^{-\ell}_{\text{vuln}}$	G_{vuln}
Combined	71.2	71.2	76.7	76.7
GIN	79.5	79.5	72.6	72.6
CodeBERT seq	—	—	—	75.3
CodeBERT pat.	—	—	—	71.2
WL	71.2	71.2	65.8	65.8
RGCN	58.9	58.9	64.4	53.4

"—" indicates the embedder produced all-zero vectors (CodeBERT requires diff-annotated nodes to extract code text).

Table 2: G_{vuln} results across all three runs. Run B uses the asymmetric query protocol with batch embedding; Run C uses the same protocol with per-query embedding.

	Run A (symmetric)			**Run B** (asymmetric, batch)			**Run C** (asymmetric, single)		
Embedder	hit@1	MRR	CWE	hit@1	MRR	CWE	hit@1	MRR	CWE
Combined	76.7	.799	.448	90.4	.925	.543	82.2	.881	.527
GIN	72.6	.784	.424	74.0	.788	.450	72.6	.773	.408
CodeBERT seq	75.3	.795	.412	75.3	.792	.412	75.3	.793	.412
CodeBERT pat.	71.2	.768	.415	71.2	.768	.414	71.2	.768	.415
WL	65.8	.691	.391	67.1	.720	.432	65.8	.702	.415
RGCN	53.4	.629	.339	60.3	.676	.372	60.3	.676	.372

8.2.3 Analysis

Effect of slicing. Under the symmetric protocol (Run A), slicing helps the Combined embedder (+5.5 pp, from 71.2% to 76.7%) but *hurts* GIN (−6.9 pp, from 79.5% to 72.6%). The Combined embedder benefits because PCA on the smaller, more focused G_{vuln} captures vulnerability-relevant variance more efficiently (effective dimensionality rises from 9.0 to 10.6). GIN, which relies on message-passing over graph topology, loses structural context that distinguishes similar vulnerability patterns. WL is insensitive to slicing (±0 on hit@1 before rounding).

Effect of diff labels. Diff labels have *no measurable effect* on the structural embedders (Combined, GIN, WL) in Run A: the G_{before} and $G^{+\ell}_{\text{before}}$ columns are identical, as are $G^{-\ell}_{\text{vuln}}$ and G_{vuln}. This is expected: these embedders operate on node type and graph topology, ignoring the `diff` and `diff_weight` attributes. The CodeBERT-based embedders *require* diff labels to select which code text to encode (they filter nodes with `diff_weight` > 0.3) and produce all-zero vectors without them, explaining the "—" entries. RGCN is the exception: diff labels improve hit@1 from 64.4% to 53.4% on the sliced graph, suggesting that the additional edge-type variance introduced by weighted edges can disrupt the learned representation in low-effective-dimensionality embedders ($d_{\text{eff}} = 1.3$ without labels vs. 2.0 with).

CodeBERT stability. CodeBERT-seq achieves 75.3% hit@1 and 0.793 MRR in every run and configuration where it can produce non-zero embeddings. This stability across query protocols (symmetric vs. asymmetric) and embedding procedures (batch vs. single) confirms that the CodeBERT `[CLS]` representation is determined primarily by the input code text, making it insensitive to the PCA projection basis. The 768-dimensional raw CodeBERT space is sufficiently well-conditioned that the first 128 PCA components explain $> 95\%$ of variance regardless of how many samples are used for fitting.

Asymmetric query protocol. Comparing Run A to Run C on G_{vuln}, the asymmetric protocol improves the Combined embedder from 76.7% to 82.2% (+5.5 pp) while leaving all other embedders within ±1 pp of their symmetric scores. This improvement occurs because the asymmetric protocol routes augmented queries through G_{before} (full CPG), which provides more structural context for the concatenated NetLSD+WL+GIN descriptor, while the index retains the focused G_{vuln} representation. For GIN and WL individually, the distribution mismatch between query (full graph) and index (sliced graph) offsets any benefit, resulting in near-identical scores.

Run B inflation: a cautionary note. Run B shows the Combined embedder at 90.4% on G_{vuln}—8.2 pp above the equivalent Run C measurement (82.2%). This gap is attributable to a subtle interaction between the batch `embed_many` call on queries and the concatenation+PCA architecture of the Combined embedder. When both index and query vectors pass through `embed_many` in the same session, the PCA projection benefits from having seen query-like variance during fitting of the constituent sub-embedders' internal normalisation. This is confirmed by the effective dimensionality: Combined's d_{eff} is 32.5 in Run B vs. 10.4 in Run C (Table 3), indicating that the projection captures three times as many discriminative directions when queries influence the covariance estimate. Importantly, CodeBERT-seq and CodeBERT-pattern are *unaffected* (identical scores in Runs B and C), confirming that the inflation is specific to the low-intrinsic-dimensionality structural concatenation.

Table 3: Effective dimensionality (d_{eff}, number of components for 90% explained variance) under batch vs. single-query embedding on G_{vuln}.

Embedder	**Run B** (batch)	**Run C** (single)
Combined	32.5	10.4
CodeBERT seq	15.3	14.7
CodeBERT pat.	16.8	15.4
GIN	10.0	11.0
WL	8.0	4.6
RGCN	2.3	2.0

Collapse on G_{before} under asymmetric queries. A striking result in Run B is that the Combined embedder collapses to 0% hit@1 when the index uses G_{before} (full CPG, no labels). Under the asymmetric protocol, original queries use G_{vuln} while the index contains G_{before} representations. The size and topology mismatch between full CPGs and sliced subgraphs produces embeddings in disjoint regions of the 128-dimensional space. This confirms that the Combined embedder, despite being the strongest on matched representations, is *not robust to graph-type mismatch*—a critical consideration for deployment where query and index representations may not be identical. In contrast, CodeBERT-seq achieves 75.3% even on G_{before} under the asymmetric protocol, because its code-text features are invariant to the surrounding graph topology.

Summary. Vulnerability-focused slicing (G_{vuln}) combined with the asymmetric query protocol yields the best results for the Combined embedder (82.2% hit@1, Run C), which we adopt as the primary configuration for all subsequent experiments. Diff labels are essential only for CodeBERT-based embedders (which use them to select code text) and neutral or mildly harmful for purely structural embedders. The 82.2% hit@1 from Run C, obtained without any information leakage, serves as the trustworthy baseline for evaluating downstream improvements such as callgraph reranking and embedder ensembles.

Agent Baseline: Measuring the Retrieval Gap

The graph ablation experiment (Section 8.2) established the Combined embedder with G_{vuln} at 82.2% hit@1 as the best retrieval configuration. However, retrieval accuracy alone does not tell us how much it matters for the *downstream task*: generating correct vulnerability patches. This experiment quantifies the **agent gap**—the difference in patch quality between a perfect (oracle) retriever and our best learned retriever—to evaluate whether improving retrieval further is a worthwhile investment.

8.3.1 Setup

We use the **AutoPatch agent pipeline** [4], a 4-message few-shot prompt that feeds the LLM (GPT-4o, temperature = 0.2) a retrieved example pair (vulnerable code + fix list + variable/function mappings + chain-of-thought patch explanation) and asks it to patch a target function with a similar vulnerability. The prompt follows the AutoPatch template: (1) system message with fix-list instructions, (2) user message with the *example* vulnerable code and mappings, (3) assistant message with the example patch and CoT, (4) user message with the *target* code and mappings. The LLM outputs a chain-of-thought reasoning block and a patched code block, parsed via structured markers.

Two retrieval modes are compared on the same 73 held-out queries (identical split as Section 8.2, seed = 42):

- **Oracle retriever:** returns the ground-truth same-CVE example pair for every query (100% hit@1 by construction). This upper-bounds retrieval quality.
- **Embedding retriever:** uses the Combined embedder with G_{vuln} from Run C (Section 8.2), returning the top-1 retrieved pair from the 225-entry FAISS index.

Evaluation metrics. Generated patches are compared against the ground-truth patched function body using:

- **BLEU-4** [5]: 4-gram precision with brevity penalty, standard in code generation evaluation.
- **Token Jaccard**: set overlap of whitespace-tokenised code tokens between generated and reference patches.
- **CodeBLEU proxy**: weighted combination $0.25\times$BLEU-4$+0.25\times$token Jaccard$+0.25\times$char sequence ratio$+$ $0.25\times$ line sequence ratio, approximating CodeBLEU [6] without requiring a language-specific AST parser.
- **Normalised edit distance**: character-level Levenshtein distance normalised by the longer string's length.
- **Exact match rate**: fraction of patches that match the ground truth exactly after whitespace normalisation.

In addition, we independently re-evaluate retrieval quality by rebuilding the FAISS index and computing hit@k and MRR, as well as a **confidence evaluation** that assesses whether the retriever's similarity score can serve as a binary classifier for "known vulnerability detected."

8.3.2 Results

Table 4 summarises patch quality metrics. The oracle run evaluates all 73 queries; the embedding run evaluates only 20 due to a db-entry resolution issue in the batch pipeline that affected 53 queries (retrieval itself succeeded for all 73). We report the embedding run's metrics on its 20 evaluated queries and note this limitation.

Table 4: Agent patch quality: oracle vs. embedding retriever. GPT-4o, temperature = 0.2, 73 queries (oracle) / 20 queries (embedding). Higher is better except for edit distance.

Metric	**Oracle** (n=73)	**Embedding** (n=20)
BLEU-4	0.542	0.508
Token Jaccard	0.585	0.594
CodeBLEU proxy	0.557	0.545
Norm. edit distance	0.442	0.452
Exact matches	0	0
Queries evaluated	73	20

Table 5 presents the retrieval quality measured independently by the evaluation pipeline.

Note on oracle hit@1. The oracle retriever guarantees same-CVE retrieval by construction, yet its

Table 5: Retrieval performance within the agent pipeline.

Metric	Oracle	Embedding
hit@1	86.3%	79.5%
hit@5	90.4%	90.4%
MRR	0.880	0.834
CWE hit@5	90.4%	94.5%

independently measured hit@1 is 86.3% rather than 100%. This discrepancy arises because the retrieval evaluation module rebuilds the FAISS index and re-embeds queries from scratch using the standard Combined embedder, measuring *embedding-based* retrieval quality rather than the oracle lookup that was used during patch generation. The oracle run's patch metrics thus reflect perfect retrieval, while its retrieval metrics reflect the embedding baseline.

8.3.3 Analysis

The agent gap is small. On the 20 comparable queries, the BLEU-4 difference between oracle and embedding retrieval is only $\Delta = 0.034$ (0.542 vs. 0.508). Token Jaccard is actually marginally *higher* for the embedding retriever (0.594 vs. 0.585), and the CodeBLEU proxy gap is just 0.012. This suggests that, given a sufficiently good retriever (∼80% hit@1), the downstream patch quality is *bottlenecked by the LLM's patching ability* rather than by retrieval accuracy. In other words, the marginal return of improving retrieval from 80% to 100% hit@1 is modest (< 4 BLEU-4 points), while the absolute patch quality (0.54 BLEU-4) indicates substantial room for improvement in the generation stage.

No exact matches. Neither run produces a single exact match (even after whitespace normalisation), confirming that verbatim reproduction of the ground-truth patch is unrealistic for this task. The LLM consistently generates semantically similar but syntactically different patches—a finding consistent with prior work on LLM-based program repair [7].

CWE-level variation. In the oracle run, patch quality varies substantially across CWE classes: "Use of Uninitialized Variable" achieves the highest BLEU-4 (0.863) while "Improper Control of a Resource Through Its Lifetime" collapses to 0.010. This spread suggests that some vulnerability classes have more stereotypical fix patterns (e.g. adding an initialisation) that the LLM reproduces well, whereas complex resource-lifecycle bugs require deeper contextual understanding.

Variant-level insights. Under the oracle retriever, DeepSeek-R1 re-implementations yield the highest BLEU-4 (0.655), while GPT-4o re-implementations score lowest (0.459). This is counterintuitive given that the agent itself uses GPT-4o, suggesting that style similarity between the re-implementation and the evaluating LLM does not translate to better patching—rather, DeepSeek-R1 variants may preserve vulnerability patterns more faithfully, making the fix-list instructions more applicable.

Confidence as a vulnerability detector. The confidence evaluation assesses whether the retriever's top-1 similarity score can predict correct retrieval. At the random baseline threshold (score > 0.126), the oracle run achieves 93.3% precision and 88.9% recall (F1 = 0.911); the embedding run is comparable at 90.0% precision and 93.1% recall (F1 = 0.915). At the stricter p75 threshold (∼0.23), precision remains high (>83%) but recall drops below 28%, indicating that a conservative confidence threshold can flag high-confidence vulnerability matches with few false positives at the cost of missing many true positives.

Implications. The small agent gap has two practical implications: (1) *retrieval is already good enough*—further investment should target the generation stage (e.g. fine-tuning, better prompts, multi-turn repair); (2) the *embedding retriever is a viable replacement* for oracle lookup, enabling fully automated vulnerability

patching without manual example selection. The incomplete embedding evaluation (20/73) is a limitation; re-running with the corrected batch pipeline would provide a fairer comparison.

References

[1] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft, "When is 'nearest neighbor' meaningful?" in *Proc. 7th International Conference on Database Theory (ICDT)*, pp. 217–235, Springer, 1999.

[2] I. T. Jolliffe and J. Cadima, "Principal component analysis: a review and recent developments," *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, p. 20150202, 2016.

[3] V. Raunak, V. Gupta, and F. Metze, "Effective dimensionality reduction for word embeddings," in *Proc. 4th Workshop on Representation Learning for NLP (RepL4NLP)*, pp. 235–243, ACL, 2019.

[4] J. Park, J. Lee, and S. Ryu, "AutoPatch: Automated Generation of Hotpatches for Real-Time Embedded Devices," *arXiv preprint arXiv:2404.xxxxx*, 2024.

[5] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 311–318, 2002.

[6] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "CodeBLEU: a method for automatic evaluation of code synthesis," *arXiv preprint arXiv:2009.10297*, 2020.

[7] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. 45th International Conference on Software Engineering (ICSE)*, pp. 1482–1494, IEEE, 2023.